



Dokumentacja projektu

Przedmiot: Zarządzanie danymi

Tytuł projektu: System rezerwacji kortów
tenisowych

Prowadzący:

inż. Eryk Chrustek

Wykonał: Miłosz Makiel 69817
Semestr, symbol kierunku i grupa: 6 IN.INP.TIM.004 A P1 2026

Rzeszów 2026r

1. Opis projektu

Projekt CourtBookingApp to aplikacja backendowa typu REST API stworzona w technologii .NET 8. Głównym celem systemu jest automatyzacja procesu rezerwacji kortów tenisowych. Aplikacja pozwala na zarządzanie użytkownikami, przeglądanie dostępnych kortów (z uwzględnieniem ich parametrów technicznych, takich jak zadaszenie) oraz składanie rezerwacji czasowych. Dane przechowywane są w chmurowej bazie danych Supabase.

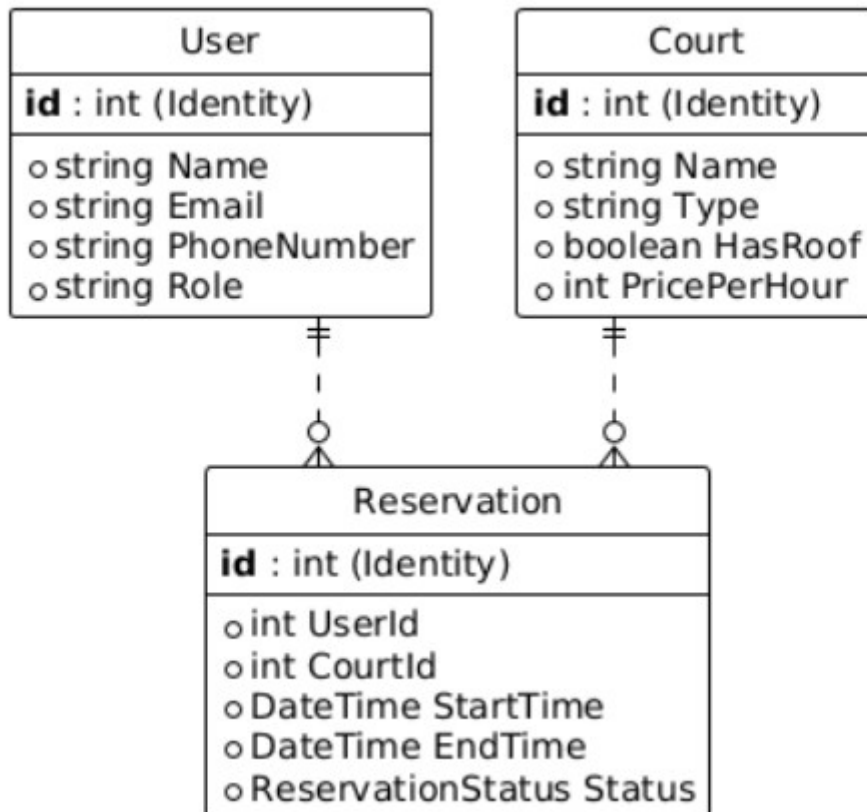
2. Wykorzystane technologie

- **Framework:** .NET 8 (ASP.NET Core Web API)
- **Baza danych:** PostgreSQL (serwowany przez platformę Supabase)
- **Integracja bazy:** Supabase-csharp / Postgrest-csharp
- **Dokumentacja:** Swagger UI
- **Język:** C#

3. Struktura bazy danych

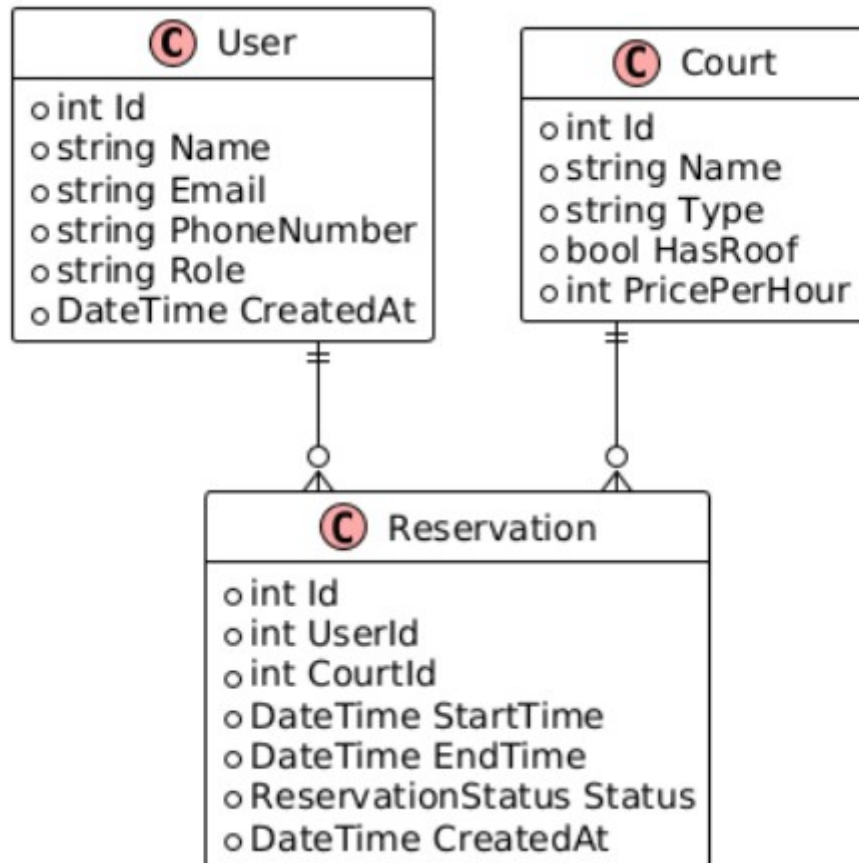
3.1 Diagram fizyczny – ERD

Poniższy diagram przedstawia relacje między tabelami w bazie PostgreSQL.



3.2 Diagram logiczny - Model domenowy

Diagram obrazujący klasy wewnątrz projektu i ich powiązania.



3.3 Opis Encji

- **User:** encja przechowująca dane użytkowników systemu, takie jak m.in. imię, nazwisko i adres e-mail. Każdy użytkownik może posiadać wiele rezerwacji, ale każda rezerwacja należy do jednego użytkownika.
- **Court:** encja reprezentująca kort dostępny do rezerwacji wraz z jego właściwościami, takimi jak typ czy cena za godzinę. Jeden kort może być rezerwowany wielokrotnie w różnych przedziałach czasowych.
- **Reservation:** encja opisująca rezerwację kortu przez użytkownika w określonym czasie. Łączy użytkownika z kortem i zawiera informacje o terminie oraz statusie rezerwacji.

Court

Reservation

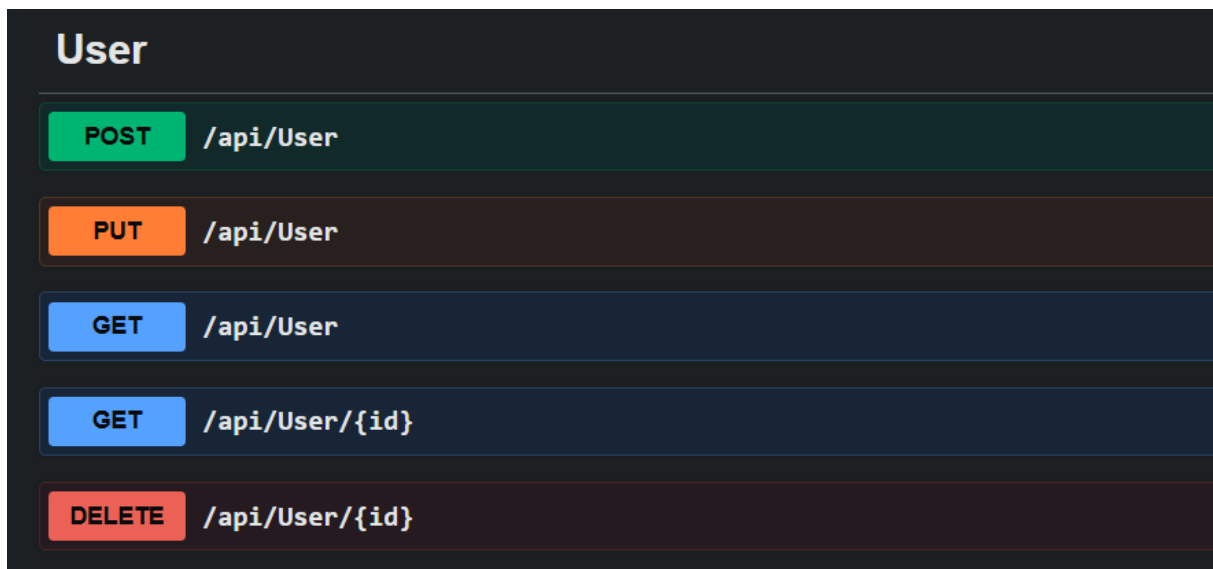
User

4. Struktura API – Swagger

4.1 Moduł User

Moduł służy do ewidencji osób korzystających z kortów.

- **POST /api/User:** Dodaje nowego użytkownika.
- **PUT /api/User/{id}:** Edytuje dane (np. zmiana maila).
- **GET /api/User:** Pobiera listę wszystkich użytkowników.
- **GET /api/User/{id}:** Pobiera rekord użytkownika.
- **DELETE /api/User/{id}:** Usuwa rekord użytkownika.



4.2 Moduł Court

Zarządzanie bazą kortów i ich dostępnością.

- **POST /api/Court:** Umożliwia dodanie nowego kortu do bazy Supabase.
- **GET /api/Court:** Wyświetla wszystkie korty.
- **DELETE /api/Court/{id}:** Usuwa kort.
- **PUT /api/Court/{id}:** Edytuje dane (np. zmiana ceny).
- **GET /api/Court/{id}:** Szczegółowe informacje o wybranym obiekcie.

Court	
POST	/api/Court
GET	/api/Court
DELETE	/api/Court
PUT	/api/Court/{id}
GET	/api/Court/{id}

4.3 Moduł Reservation

Kluczowy moduł obsługujący proces rezerwacji.

- **POST /api/Reservation:** Tworzy nową rezerwację.
- **GET /api/Reservation:** Podgląd wszystkich dokonanych rezerwacji.
- **GET /api/Reservation/{id}:** Podgląd szczegółów konkretnej rezerwacji.
- **PATCH /api/Reservation/{id}:** Anuluje rezerwację.
- **DELETE /api/Reservation/{id}:** Usuwa rezerwację.

Reservation	
POST	/api/Reservation
GET	/api/Reservation
GET	/api/Reservation/{id}
PATCH	/api/Reservation/{id}
DELETE	/api/Reservation/{id}

5. Architektura i Przepływ Danych

5.1 Architektura

Aplikacja wykorzystuje architekturę REST API. Kontrolery .NET Core pośredniczą w komunikacji między użytkownikiem a API Supabase. Wykorzystano wstrzykiwanie zależności (Dependency Injection) do zarządzania połączeniem z bazą danych.

5.2 Przepływ danych

1. Użytkownik wysyła żądanie przez Swaggera

Użytkownik wykonuje żądanie HTTP do endpointu API za pomocą interfejsu Swagger.

2. Kontroler .NET waliduje dane

Kontroler odbiera żądanie, mapuje dane i sprawdza ich poprawność.

3. Service przetwarza logikę biznesową

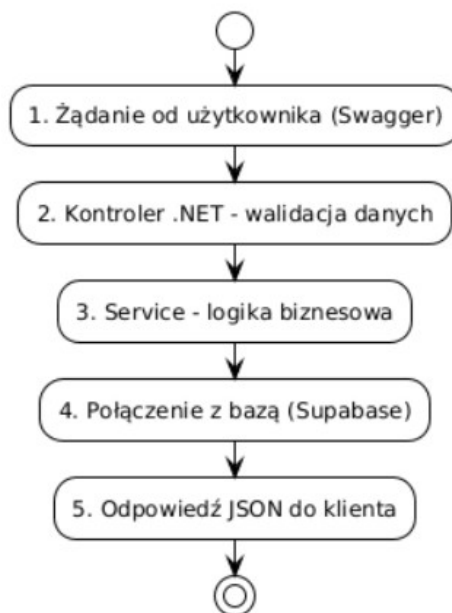
Warstwa serwisowa realizuje logikę aplikacji i przygotowuje operacje na danych.

4. Połączenie z bazą (Supabase)

Serwis wykonuje asynchroniczne zapytanie do bazy danych w Supabase.

5. Odpowiedź JSON do klienta

Wynik z bazy wraca przez backend i jest zwracany do klienta w formacie JSON.



6. Repozytorium

- Link do repozytorium GitHub: <https://github.com/MagicJJ34/CourtBookingApp>